



**SRI RAMAKRISHNA
ENGINEERING COLLEGE**

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACTIVE LEARNING METHODOLOGY REPORT

REMOTE DATA LOGGER USING LPC2148

71812401017 – ANIRUDH R

71812401018 – ARUN KHANDASAMY A P

71812401019 – ARUN SOUNDAPPAN

71812401020 – ASHWIN KUMAR N

71812401021 – ASHWIN S

THIRD YEAR B.E. CSE – V SEM

Academic Year 2026-2027

20EC252 & MICROCONTROLLERS AND PROGRAMMING

COURSE COORDINATOR

Mrs.M.Amuthasurabi, AP/CSE



**SRI RAMAKRISHNA
ENGINEERING COLLEGE**

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,

Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20EC252 & MICROCONTROLLERS AND PROGRAMMING

Assessment

INDEX

S. No.	Name & Roll No.	Tool Explanation (2)	Implementation (5)	Presentation (3)	Total (10)
1.	71812401017 – ANIRUDH R				
2.	71812401018 – ARUN KHANDASAMY A P				
3.	71812401019 – ARUN SOUNDAPPAN				
4.	71812401020 – ASHWIN KUMAR N				
5.	71812401021 – ASHWIN S				

SIGNATURE OF COURSE COORDINATOR
Mrs.M.Amuthasurabi, AP/CSE



SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	INTRODUCTION	4
2.	WORKFLOW DESCRIPTION	5
3.	IMPLEMENTATION STEPS	6
4.	CODE IMPLEMENTATION	8
5.	OUTPUT	13
6.	DIAGRAMMATIC REPRESENTATION	17
7.	CONCLUSION	19

1. INTRODUCTION

A **data logger** is an electronic system that automatically measures one or more physical quantities, converts the measurements into digital form and records or transmits them at fixed intervals without continuous human supervision. Typical uses are in greenhouses, cold-storage rooms, weather stations, laboratories and industrial plants, wherever parameters such as temperature, light intensity or soil moisture must be observed over a long period.

A **remote data logger** extends this idea by separating the place where the data is measured from the place where it is analysed. The sensing unit is installed at the monitored location, while the readings are sent to a computer through a communication link so that an operator can view, store and process them without being present at the site. Continuous monitoring is necessary because many physical parameters change slowly and unpredictably; a periodic record makes it possible to detect abnormal conditions, study trends and take corrective action in time.

The sensors used for such parameters usually produce a continuously varying **analog voltage**, whereas a microcontroller can process only digital numbers. An analog-to-digital converter (ADC) therefore forms the bridge between the sensors and the processor. In this ALM activity the remote data logger is implemented using the **LPC2148**, a 32-bit microcontroller from NXP based on the ARM7TDMI-S core. Its main features are listed in Table 1. Because the ADC, UART and timers are on the chip, no external converter or communication IC is needed apart from the sensors and a serial-level interface.

Table 1: Relevant specifications of the LPC2148

Feature	Specification
Processor core	32-bit ARM7TDMI-S RISC core with ARM and Thumb instruction sets
Clock	Up to 60 MHz CCLK using the on-chip PLL (12 MHz crystal in this design)
On-chip memory	512 kB flash program memory; 32 kB static RAM plus 8 kB RAM shared with USB
ADC	Two 10-bit successive-approximation ADCs (ADC0 and ADC1), 14 analog input channels in total
UART	Two 16C550-compatible UARTs (UART0 and UART1) with 16-byte FIFOs
Timers	Two 32-bit timers/counters (Timer0 and Timer1) with prescaler, match and capture registers
Interrupts	Vectored Interrupt Controller (VIC) with 16 vectored IRQ slots
GPIO / supply	Up to 45 general-purpose I/O pins; single 3.3 V supply, 3.3 V logic levels

The proposed system uses three on-chip modules of the LPC2148 working together. The **ADC** samples the voltages from three analog sensors (an LM35 temperature sensor, an LDR light sensor and a soil-moisture sensor) connected to separate input channels. **Timer0** generates an interrupt every second so that the readings are taken periodically and not at random. **UART0** then transmits the processed readings as text to a PC, where they are displayed on a serial terminal. The complete program is written in Embedded C and built with Keil μ Vision; the generated HEX file is loaded into a Proteus simulation of the LPC2148 whose virtual terminal displays the UART output.

Problem statement: “A remote data logger is to be implemented using the LPC2148 microcontroller. The system should collect data from multiple analog sensors and send it to a PC via UART at regular intervals. Outline the system design, emphasizing ADC configuration, UART communication, and timer usage in LPC2148.”

Objective. The objective of this report is to design and implement the remote data logger so that it can:

- acquire data from three analog sensors using the 10-bit ADC of the LPC2148;
- send the readings to a PC through UART0 at 9600 baud;

- trigger every acquisition at a fixed one-second interval using Timer0; and
- build the program in Keil μ Vision and verify the UART output in a simulation.

2. WORKFLOW DESCRIPTION

The remote data logger works as a continuous acquire–process–transmit cycle that is paced by Timer0. The complete workflow is shown in Figure 1; each numbered stage is explained below.

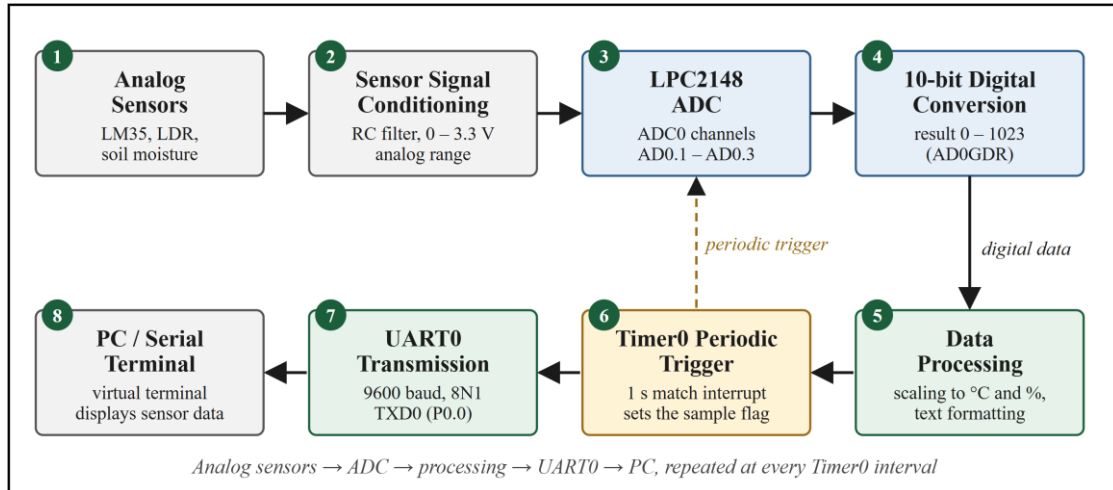


Figure 1: Overall Workflow of the Remote Data Logger Using LPC2148

1. **Analog sensors.** Three sensors measure the quantities to be logged: an LM35 temperature sensor (10 mV per °C), a light-dependent resistor (LDR) connected as a voltage divider, and a soil-moisture sensor with an analog output. Each produces a voltage that varies continuously with the measured quantity.
2. **Sensor signal conditioning.** The sensor outputs are brought into the 0 – 3.3 V range accepted by the LPC2148 analog pins. A series resistor and a small capacitor at each input (for example 1 k Ω and 0.1 μ F) form a low-pass filter that suppresses noise. The analog inputs are not 5 V tolerant and must never exceed VREF.
3. **LPC2148 ADC.** ADC0 is a 10-bit successive-approximation converter. The sensors are connected to channels AD0.1, AD0.2 and AD0.3 (pins P0.28, P0.29 and P0.30), and an analog multiplexer connects one channel at a time to the converter (Table 2).
4. **10-bit digital conversion.** Each conversion returns a number between 0 and 1023 in the RESULT field of AD0GDR, proportional to the input voltage: $V_{in} = \text{ADC} \times 3.3 / 1023$.
5. **Data processing.** The CPU converts each count into an engineering value (tenths of °C for the LM35, percentage for the LDR and soil moisture) using integer arithmetic and prepares the ASCII text lines for transmission.
6. **Timer0 periodic trigger.** Timer0 is programmed to match every second and raises an interrupt. The interrupt routine only sets a flag; the main program then performs one complete acquisition cycle, so the sampling is exactly periodic and the processor is free between samples.
7. **UART0 transmission.** The text is written one character at a time to the transmit holding register U0THR at 9600 baud, 8 data bits, no parity and 1 stop bit, on pin P0.0 (TXD0).
8. **PC / serial terminal.** A serial-level converter (or the virtual terminal in the simulation) receives the data, and the terminal window displays the sample number and the three readings. The program then waits for the next Timer0 interrupt and the cycle repeats.

Table 2: Sensor and ADC channel mapping

Sensor	Parameter	Pin	ADC channel	Output and conversion
LM35	Temperature	P0.28	AD0.1	10 mV per °C; $T = V_{in} \div 10 \text{ mV}$
LDR with 10 k Ω divider	Light intensity	P0.29	AD0.2	0 – 3.3 V; level (%) = $ADC \times 100 / 1023$
Soil-moisture sensor	Soil moisture	P0.30	AD0.3	0 – 3.3 V; moisture (%) = $ADC \times 100 / 1023$

3. IMPLEMENTATION STEPS

The implementation of the remote data logger is carried out in Keil μ Vision for program development and compilation. The workflow covers creating the LPC2148 project, selecting the device, adding the source file, configuring the target, building the project, generating the HEX file and finally using the HEX file in the Proteus simulation. The steps are summarised in Table 3 and described in Sections 3.1 to 3.14; the screenshots for these steps are given in Section 5 (Figures 2 to 12).

Table 3: Summary of implementation steps

Step	Activity	Tool / menu
3.1	Open Keil μ Vision	μ Vision 5 (MDK-ARM)
3.2	Create a new project	Project $\rightarrow$ New μ Vision Project
3.3	Select the LPC2148 device	Select Device for Target
3.4	Add the main.c source file	Source Group 1 $\rightarrow$ Add New Item
3.5	Write the Embedded C program	μ Vision editor
3.6	Configure target options	Project $\rightarrow$ Options for Target $\rightarrow$ Target
3.7	Build the project	Project $\rightarrow$ Build Target (F7)
3.8	Generate the HEX file	Options for Target $\rightarrow$ Output
3.9	Configure the simulation	Proteus ISIS, LPC2148 properties
3.10	Configure the sensor inputs	Potentiometers on P0.28 – P0.30
3.11	Configure UART / virtual terminal	Virtual Terminal on P0.0 / P0.1
3.12	Run the simulation	Proteus Play button
3.13	Observe the UART output	Virtual Terminal window
3.14	Project completion	Project files and HEX file

3.1 Open Keil μ Vision

The installed Keil μ Vision 5 development environment is opened. The MDK-ARM edition is used because the LPC2148 is an ARM7-based microcontroller. The Project window, the editor area and the Build Output window are displayed.

3.2 Create a New Project

A new project is created using Project $\rightarrow$ New μ Vision Project. A suitable project name such as Remote_Data_Logger is entered, and a separate folder is selected for the project files (Figure 2). Keeping each project in its own folder keeps the source, object and HEX files organised.

3.3 Select LPC2148

After the project is saved, the Select Device for Target window opens. LPC2148 is searched and selected under the NXP (founded by Philips) LPC2000 family, and OK is clicked (Figure 3). Selecting the correct device makes μ Vision use the right registers, memory map and compiler

settings. When μ Vision asks whether to copy the startup file Startup.s to the project folder and add it to the project, Yes is chosen; Startup.s initialises the stacks and the vector table.

3.4 Add main.c

In the Project window, Source Group 1 is right-clicked and Add New Item to Group 'Source Group 1' is chosen. A C File (.c) is selected and saved with the name main (Figure 4). This file contains the complete Embedded C program for the ADC, UART0 and Timer0.

3.5 Write the Embedded C Program

The program of Section 4 is typed into main.c and saved with Ctrl + S (Figure 5). It includes the header file LPC214x.h to access the LPC2148 registers, configures the PLL, GPIO, ADC channels AD0.1 to AD0.3, UART0 and Timer0, and contains the Timer0 interrupt routine, the sampling routine log_sample() and the main loop.

3.6 Configure Target Options

Project → Options for Target is opened (Alt + F7). On the Target tab the device is verified and the crystal frequency is entered as 12.000 MHz, matching the hardware assumption (Figure 6). The memory areas selected for the LPC2148 (on-chip flash and RAM) are accepted. If the standard library is not required, the Use MicroLIB option may be ticked to reduce the program size; sprintf() with integer formats is supported.

3.7 Build the Project

The project is compiled with Project → Build Target or by pressing F7. μ Vision compiles main.c and Startup.s, links the output and checks the program for syntax and declaration errors. The Build Output window shows the compilation status, the program size, and the numbers of errors and warnings (Figure 7). A successful build should report 0 errors; any warnings are checked and corrected where necessary.

3.8 Generate HEX File

To create the file needed by the simulation, Options for Target → Output is opened and Create HEX File is ticked (Figure 8). After clicking OK, the project is rebuilt and μ Vision generates Remote_Data_Logger.hex in the project's output folder (Objects). The file contains the machine code of the program in Intel HEX format.

3.9 Configure Simulation

The simulation is carried out in Proteus. An LPC2148 microcontroller is placed in the schematic together with the sensor sources and a virtual terminal (Figure 9). The properties of the LPC2148 component are opened by double-clicking it. In the Program File field the HEX file generated by Keil is selected, and the crystal frequency is set to 12 MHz. The supply pins, including VDDA and VREF (+3.3 V) and the ground pins, are connected so that the ADC has a 3.3 V reference.

3.10 Configure Sensor Inputs

Each analog sensor is represented in the simulation by a variable voltage source. A potentiometer between +3.3 V and ground (or the corresponding sensor model) is connected to each ADC pin, as listed in Table 4. The potentiometer setting sets the sensor output voltage; for example 0.30 V on the LM35 input represents 30 °C. Changing the settings during the run changes the readings in the following samples.

Table 4: Connections used for the simulation

Proteus component	Connected to	Represents / setting
Potentiometer or LM35 model	P0.28 (AD0.1)	LM35 output, 10 mV per °C (0.30 V = 30 °C)
Potentiometer or LDR divider	P0.29 (AD0.2)	Light sensor output, 0 – 3.3 V
Potentiometer	P0.30 (AD0.3)	Soil-moisture sensor output, 0 – 3.3 V
+3.3 V supply	VDDA, VREF, VDD pins	ADC reference and supply
Virtual Terminal RXD	P0.0 (TXD0)	Receives the data sent by UART0
Virtual Terminal TXD	P0.1 (RXD0)	Connected for completeness; not used by the program

3.11 Configure UART / Virtual Terminal

The Virtual Terminal is connected to UART0: its RXD input to P0.0 (TXD0) and its TXD output to P0.1 (RXD0). Its properties are set to match the program: baud rate 9600, 8 data bits, no parity and 1 stop bit. The terminal window opens automatically when the simulation starts.

3.12 Run the Simulation

After the HEX file is loaded and the connections are checked, the Play button in Proteus is pressed. The LPC2148 starts executing the program: the PLL, GPIO, ADC, UART0 and Timer0 are initialised, the start-up banner is sent, and from then on Timer0 triggers one logging cycle every second.

3.13 Observe UART Output

The Virtual Terminal first shows the banner REMOTE DATA LOGGER. After every timer interval a block is added containing the sample number, the temperature in °C, the light level in %, the soil moisture in % and the message “Data Logged Successfully” (Figures 10 to 12). When a potentiometer setting is changed, the corresponding reading changes in the next block.

3.14 Project Completion

After successful compilation and simulation, the implementation is complete. The project contains the LPC2148 target, the Embedded C source file main.c, the generated HEX file and the Proteus simulation. Correct periodic display of the three sensor readings on the virtual terminal verifies the ADC, UART0 and Timer0 operation of the remote data logger.

4. CODE IMPLEMENTATION

The software of the remote data logger is developed in Embedded C using Keil µVision. The program is written specifically for the LPC2148 and uses the header file LPC214x.h for access to the special function registers. It assumes a 12 MHz crystal, CCLK = 60 MHz and PCLK = 15 MHz. Sections 4.1 to 4.8 describe the individual parts of the program and Section 4.9 gives the complete program.

4.1 Pin Configuration

Table 5: Pin configuration of the remote data logger

Function	LPC2148 pin	Purpose
LM35 temperature sensor	P0.28 / AD0.1	Analog input 1, temperature
LDR light sensor	P0.29 / AD0.2	Analog input 2, light level
Soil-moisture sensor	P0.30 / AD0.3	Analog input 3, soil moisture
UART TX	P0.0 / TXD0	Transmits the sensor data to the PC
UART RX	P0.1 / RXD0	Receive line of UART0 (configured, not used)
Activity LED (optional)	P0.7	Switched on while one sample is acquired and sent

4.2 ADC Initialization

PINSEL1 is first cleared and then written so that P0.28, P0.29 and P0.30 become the analog inputs AD0.1, AD0.2 and AD0.3. AD0CR powers up the converter (PDN = 1) and sets CLKDIV = 3, giving an ADC clock of 15 MHz / 4 = 3.75 MHz, below the 4.5 MHz limit (Table 6).

```
void adc_init(void)
{
    PINSEL1 &= ~((3u << 24) | (3u << 26) | (3u << 28));
    PINSEL1 |= ((1u << 24) | (1u << 26) | (1u << 28)); /* AD0.1, AD0.2, AD0.3 */
    AD0CR = (1u << 21) | (3u << 8); /* PDN = 1, CLKDIV = 3 (3.75 MHz) */
}
```

Table 6: ADC configuration parameters

Parameter	Register / bits	Value used
Pin function	PINSEL1 bits 29:24	01 for P0.28, P0.29, P0.30
Operating mode	AD0CR.PDN (bit 21)	1 (converter operational)
ADC clock	AD0CR.CLKDIV (bits 15:8)	3 → 3.75 MHz
Conversion mode	AD0CR.BURST (bit 16)	0 (software controlled)
Resolution and time	AD0CR.CLKS (bits 19:17)	000 → 10 bits, 11 clocks
Channel select	AD0CR.SEL (bits 7:0)	0x02, 0x04, 0x08 (AD0.1, AD0.2, AD0.3)
Result	AD0GDR (RESULT 15:6, DONE 31)	0 – 1023

4.3 ADC Reading

adc_read() selects one channel through the SEL field, starts a conversion with START = 001, waits for the DONE flag (bit 31 of AD0GDR) and returns the 10-bit RESULT field (bits 15:6). The START bits are then cleared so that the channel can be changed safely for the next conversion. A conversion takes 11 ADC clocks, about 2.9 μs.

```
unsigned int adc_read(unsigned char ch)
{
    unsigned int r;
    AD0CR = (AD0CR & ~0xFFu) | (1u << ch); /* select channel (SEL field) */
    AD0CR |= (1u << 24); /* START = 001: start conversion */
    do { r = AD0GDR; } while (!(r & 0x80000000u)); /* wait for DONE flag */
    AD0CR &= ~(7u << 24); /* stop conversion */
    return (r >> 6) & 0x3FF; /* 10-bit RESULT (bits 15:6) */
}
```

4.4 UART Initialization

UART0 is configured for 9600 baud, 8 data bits, no parity and 1 stop bit (Table 7). PINSEL0 bits 3:0 are set to 0101 so that P0.0 is TXD0 and P0.1 is RXD0. With DLAB = 1, the divisor PCLK /

$(16 \times 9600) = 97.7$, rounded to 98, is written to U0DLL; the actual baud rate is 9566, an error of 0.35 %.

```
void uart0_init(void)
{
    PINSEL0 = (PINSEL0 & ~0x0Fu) | 0x05u;    /* P0.0 = TXD0, P0.1 = RXD0    */
    U0LCR = 0x83;                            /* 8N1, DLAB = 1            */
    U0DLL = ((PCLK_HZ / 16) + (BAUD / 2)) / BAUD; /* divisor = 98            */
    U0DLM = 0;
    U0LCR = 0x03;                            /* DLAB = 0                 */
    U0FCR = 0x07;                            /* enable and reset FIFOs   */
}
```

Table 7: UART0 configuration parameters

Parameter	Setting	Register / value
Baud rate	9600	U0DLL = 98, U0DLM = 0
Data bits / parity / stop	8 / none / 1	U0LCR = 0x03 (after DLAB = 0)
Pins	TXD0 = P0.0, RXD0 = P0.1	PINSEL0 = 0x05
FIFO	Enabled and reset	U0FCR = 0x07
Transmit status	THRE flag	U0LSR bit 5

4.5 UART Transmission

uart0_putc() waits until the transmit holding register is empty (THRE = 1 in U0LSR) and then writes the character to U0THR. uart0_puts() sends a complete string by calling uart0_putc() for every character.

```
void uart0_putc(char c)
{
    while (!(U0LSR & 0x20));                /* wait until THR is empty */
    U0THR = c;
}
void uart0_puts(const char *s)
{
    while (*s)
        uart0_putc(*s++);
}
```

4.6 Timer0 Initialization

Timer0 counts PCLK cycles in hardware. T0PR = 14999 makes the timer counter increment every 1 ms, and T0MR0 = 999 produces a match after 1000 ticks, which is 1 s. T0MCR = 0x03 generates an interrupt and resets the counter at every match. Timer0 is source 4 of the VIC and is assigned to vectored slot 0 (Table 8).

```
void timer0_init(void)
{
    T0TCR = 0x02;                            /* reset counter            */
    T0PR = (PCLK_HZ / 1000) - 1;              /* 14999 -> 1 ms per tick   */
    T0MR0 = SAMPLE_MS - 1;                   /* 999 -> match every 1 s   */
    T0MCR = 0x03;                            /* interrupt + reset on MRO  */
    VICVectAddr0 = (unsigned long)timer0_isr;
    VICVectCntl0 = 0x20 | 4;                  /* slot 0 enabled, Timer0 = 4 */
    VICIntEnable = (1u << 4);
    T0TCR = 0x01;                            /* start Timer0             */
}
```

Table 8: Timer0 configuration parameters

Parameter	Register	Value	Effect
Peripheral clock	VPBDIV	0x00	PCLK = CCLK/4 = 15 MHz
Prescaler	T0PR	14999	1 ms per timer tick
Match value	T0MR0	999	1 s sampling interval
Match control	T0MCR	0x03	Interrupt and reset on MR0
VIC slot	VICVectCntl0	0x24	Slot 0 enabled, source 4 (Timer0)
Timer control	T0TCR	0x01	Counter enabled

4.7 Timer0 Interrupt

The interrupt service routine is kept very short: it sets `sample_flag`, clears the MR0 interrupt flag by writing 1 to T0IR, and writes 0 to VICVectAddr to signal the end of the interrupt. The keyword `__irq` makes the ARM compiler generate the interrupt entry and exit code. The time-consuming work is done in the main loop, not inside the interrupt.

```
__irq void timer0_isr(void)
{
    sample_flag = 1;                /* 1 s has elapsed          */
    T0IR = 0x01;                    /* clear MR0 interrupt flag */
    VICVectAddr = 0;                /* end of interrupt         */
}
```

4.8 Data Processing

`log_sample()` reads the three channels in turn and converts the counts with integer arithmetic only: $a1 \times 3300 / 1023$ gives millivolts, which for the LM35 (10 mV per °C) equals tenths of a degree Celsius; the LDR and soil-moisture counts are scaled to a percentage of full scale. The values are formatted with `sprintf()` and sent through UART0. The activity LED on P0.7 is on while the sample is being acquired and sent.

```
void log_sample(void)
{
    char buf[40];
    unsigned int a1, a2, a3, t10, light, moist;
    IO0SET = LED_PIN;                /* activity LED on          */
    a1 = adc_read(1);                /* LM35 (AD0.1)             */
    a2 = adc_read(2);                /* LDR (AD0.2)              */
    a3 = adc_read(3);                /* soil moisture (AD0.3)    */
    t10 = (a1 * 3300u) / 1023u;      /* mV = tenths of deg C (10 mV/C) */
    light = (a2 * 100u) / 1023u;     /* percent of full scale    */
    moist = (a3 * 100u) / 1023u;     /* percent of full scale    */
    sample_no++;
    sprintf(buf, "Sample No      : %03u\r\n", sample_no);
    uart0_puts(buf);
    sprintf(buf, "Temperature   : %u.%u C\r\n", t10 / 10, t10 % 10);
    uart0_puts(buf);
    sprintf(buf, "Light        : %u %%\r\n", light);
    uart0_puts(buf);
    sprintf(buf, "Soil Moisture : %u %%\r\n", moist);
    uart0_puts(buf);
    uart0_puts("-----\r\n");
    uart0_puts("Data Logged Successfully\r\n\r\n");
    IO0CLR = LED_PIN;                /* activity LED off         */
}
```

4.9 Complete Program

The complete program initialises the PLL, GPIO, ADC, UART0 and Timer0 in that order, sends the start-up banner and then enters an endless loop. The loop checks the volatile flag `sample_flag`, which Timer0 sets every second; when it is set, the flag is cleared and `log_sample()` reads the three sensors and transmits the results.

```
/* Remote Data Logger using LPC2148 - main.c (Keil uVision, Embedded C) */
#include <LPC214x.h>
#include <stdio.h>
#define PCLK_HZ      15000000UL      /* PCLK = CCLK / 4 = 15 MHz */
#define BAUD         9600UL
#define SAMPLE_MS    1000UL          /* logging interval = 1 s */
```

```

#define LED_PIN      (1u << 7)          /* P0.7: optional activity LED          */
volatile unsigned char sample_flag = 0; /* set by the Timer0 interrupt          */
unsigned int sample_no = 0;
/* ----- clock ----- */
void pll_init(void)
{
    unsigned int t = 100000;             /* lock time-out counter                */
    PLL0CFG = 0x24;                      /* M = 5, P = 2 -> CCLK = 60 MHz        */
    PLL0CON = 0x01;                      /* enable PLL                           */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    while (!(PLL0STAT & (1u << 10)) && --t); /* wait for PLL lock (time-out)        */
    PLL0CON = 0x03;                      /* enable and connect PLL               */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    MAMTIM = 0x03;                      /* flash accelerator timing             */
    MAMCR = 0x02;                      /* fully enabled                       */
    VPBDIV = 0x00;                      /* PCLK = CCLK / 4 = 15 MHz            */
}
/* ----- GPIO ----- */
void gpio_init(void)
{
    IO0DIR |= LED_PIN;                  /* P0.7 as output                       */
    IO0CLR = LED_PIN;                  /* LED off                              */
}
/* ----- ADC ----- */
void adc_init(void)
{
    PINSEL1 &= ~( (3u << 24) | (3u << 26) | (3u << 28) );
    PINSEL1 |= ( (1u << 24) | (1u << 26) | (1u << 28) ); /* AD0.1, AD0.2, AD0.3 */
    AD0CR = (1u << 21) | (3u << 8);      /* PDN = 1, CLKDIV = 3 (3.75 MHz) */
}
unsigned int adc_read(unsigned char ch)
{
    unsigned int r;
    AD0CR = (AD0CR & ~0xFFu) | (1u << ch); /* select channel (SEL field)          */
    AD0CR |= (1u << 24);                  /* START = 001: start conversion      */
    do { r = AD0GDR; } while (!(r & 0x80000000u)); /* wait for DONE flag                  */
    AD0CR &= ~(7u << 24);                /* stop conversion                     */
    return (r >> 6) & 0x3FF;             /* 10-bit RESULT (bits 15:6)          */
}
/* ----- UART0 ----- */
void uart0_init(void)
{
    PINSEL0 = (PINSEL0 & ~0x0Fu) | 0x05u; /* P0.0 = TXD0, P0.1 = RXD0            */
    U0LCR = 0x83;                        /* 8N1, DLAB = 1                      */
    U0DLL = ((PCLK_HZ / 16) + (BAUD / 2)) / BAUD; /* divisor = 98                      */
    U0DLM = 0;
    U0LCR = 0x03;                        /* DLAB = 0                            */
    U0FCR = 0x07;                        /* enable and reset FIFOs              */
}
void uart0_putc(char c)
{
    while (!(U0LSR & 0x20));             /* wait until THR is empty             */
    U0THR = c;
}
void uart0_puts(const char *s)
{
    while (*s)
        uart0_putc(*s++);
}
/* ----- Timer0 ----- */
__irq void timer0_isr(void)
{
    sample_flag = 1;                    /* 1 s has elapsed                     */
    T0IR = 0x01;                       /* clear MR0 interrupt flag            */
    VICVectAddr = 0;                   /* end of interrupt                    */
}
void timer0_init(void)
{
    T0TCR = 0x02;                      /* reset counter                       */
    T0PR = (PCLK_HZ / 1000) - 1;        /* 14999 -> 1 ms per tick              */
    T0MR0 = SAMPLE_MS - 1;             /* 999 -> match every 1 s              */
    T0MCR = 0x03;                      /* interrupt + reset on MR0            */
    VICVectAddr0 = (unsigned long)timer0_isr;
    VICVectCntl0 = 0x20 | 4;           /* slot 0 enabled, Timer0 = 4          */
    VICIntEnable = (1u << 4);          /* start Timer0                        */
    T0TCR = 0x01;
}
/* ----- acquisition, processing, output ----- */
void log_sample(void)
{
    char buf[40];
}

```

```

unsigned int a1, a2, a3, t10, light, moist;
IO0SET = LED_PIN;                                /* activity LED on */
a1 = adc_read(1);                                /* LM35 (AD0.1) */
a2 = adc_read(2);                                /* LDR (AD0.2) */
a3 = adc_read(3);                                /* soil moisture (AD0.3) */
t10 = (a1 * 3300u) / 1023u;                       /* mV = tenths of deg C (10 mV/C) */
light = (a2 * 100u) / 1023u;                      /* percent of full scale */
moist = (a3 * 100u) / 1023u;                      /* percent of full scale */
sample_no++;
sprintf(buf, "Sample No : %03u\r\n", sample_no);
uart0_puts(buf);
sprintf(buf, "Temperature : %u.%u C\r\n", t10 / 10, t10 % 10);
uart0_puts(buf);
sprintf(buf, "Light : %u %%\r\n", light);
uart0_puts(buf);
sprintf(buf, "Soil Moisture : %u %%\r\n", moist);
uart0_puts(buf);
uart0_puts("-----\r\n");
uart0_puts("Data Logged Successfully\r\n\r\n");
IO0CLR = LED_PIN;                                /* activity LED off */
}
/* ----- main ----- */
int main(void)
{
    pll_init();
    gpio_init();
    adc_init();
    uart0_init();
    timer0_init();
    uart0_puts("=====\r\n");
    uart0_puts("REMOTE DATA LOGGER\r\n");
    uart0_puts("=====\r\n");
    while (1)
    {
        if (sample_flag)                          /* set every second by Timer0 */
        {
            sample_flag = 0;
            log_sample();
        }
    }
}

```

5. OUTPUT

This section presents the screenshots of the Keil μ Vision implementation and of the simulation of the remote data logger: project creation, device selection, source file, program, target configuration, successful build, HEX file generation, simulation setup and the UART output received on the virtual terminal. Each figure is followed by an explanation of what it demonstrates.

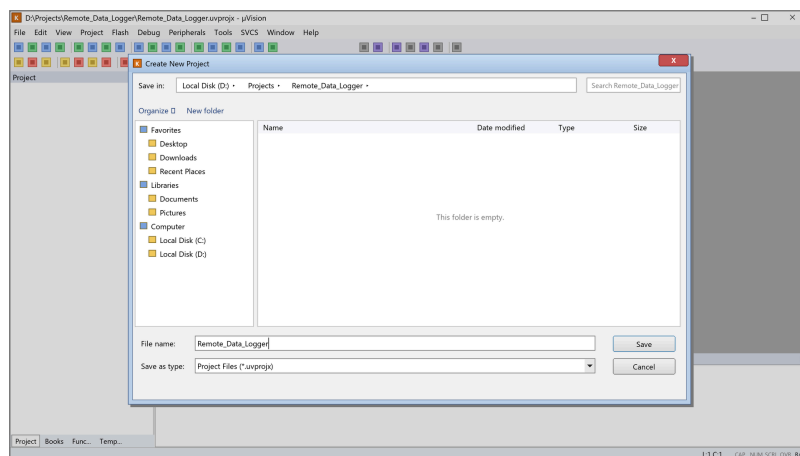


Figure 2: Creating a New μ Vision Project

Figure 2 shows the new project being created and saved in its own folder.

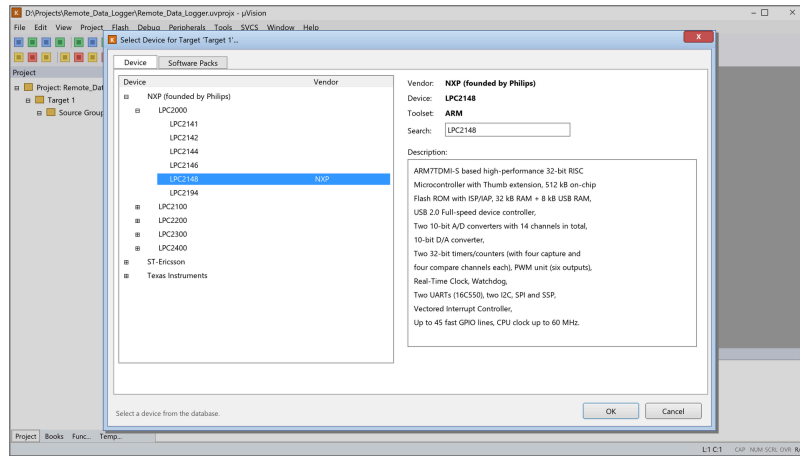


Figure 3: Selecting LPC2148 Device

Figure 3 shows the LPC2148 device selected for the target.

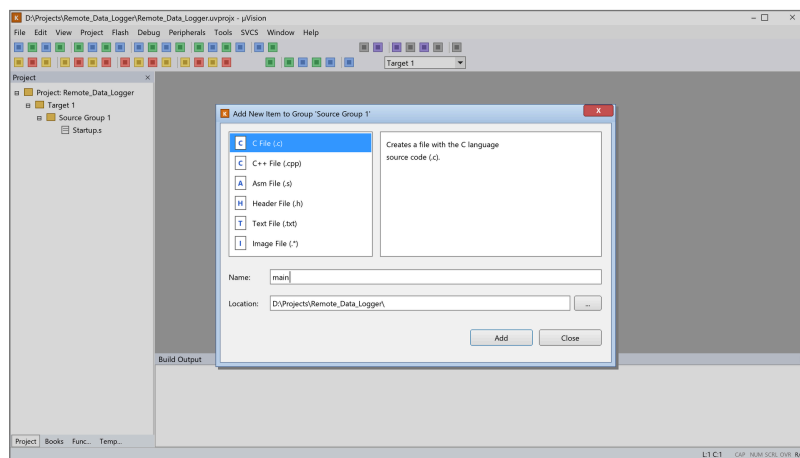


Figure 4: Adding main.c Source File

Figure 4 shows the main.c source file being added to Source Group 1.

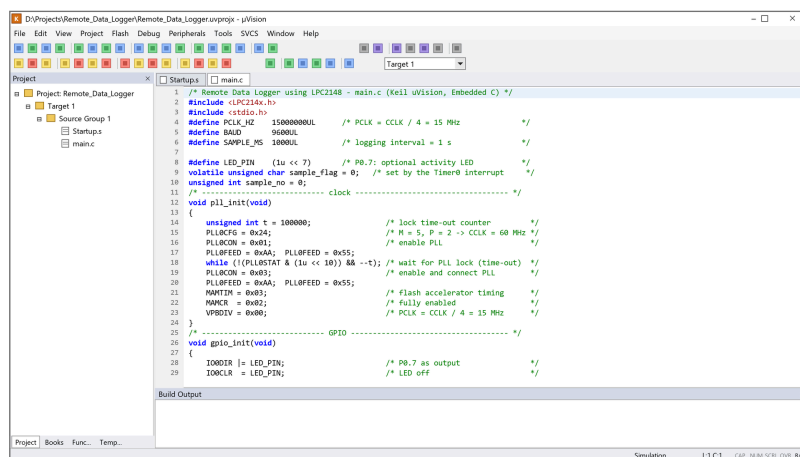


Figure 5: Embedded C Program in Keil uVision

Figure 5 shows the Embedded C program of the remote data logger in the uVision editor.

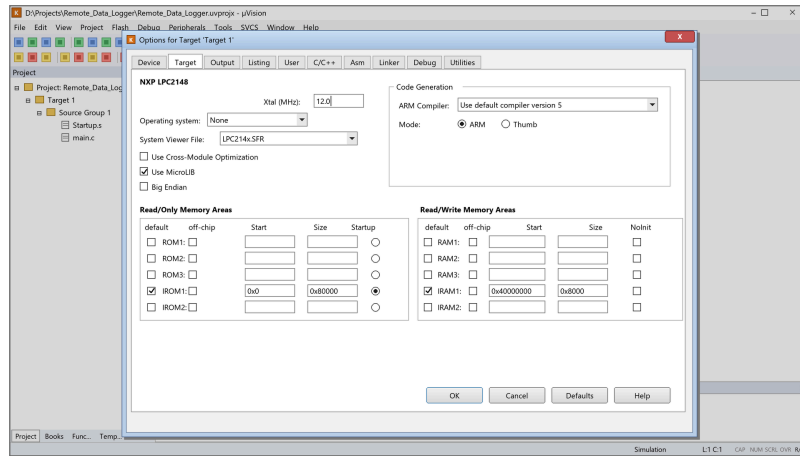


Figure 6: Target Configuration for LPC2148

Figure 6 shows the target options configured for the LPC2148 with a 12 MHz crystal.

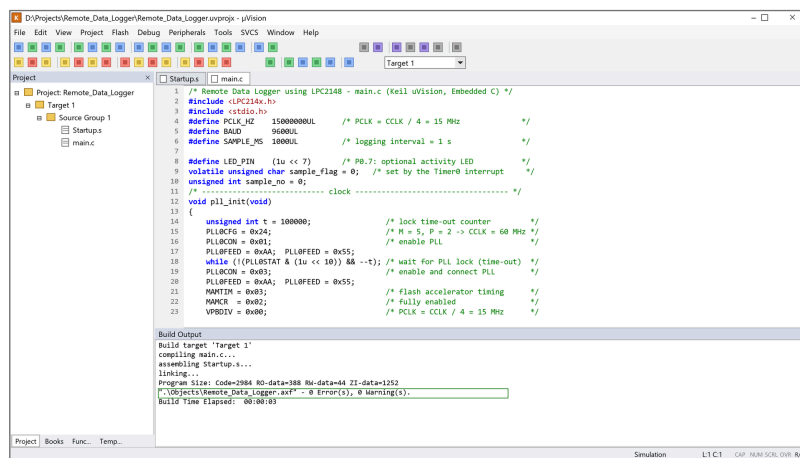


Figure 7: Successful Build Output

Figure 7 shows the build output of the project. A successful compilation reports 0 errors and the program size, confirming that the Embedded C program is correct for the LPC2148.

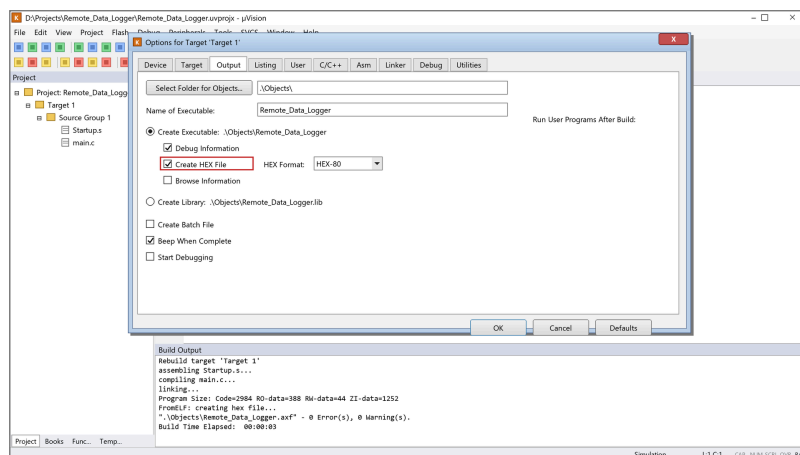


Figure 8: HEX File Generation

Figure 8 shows the Create HEX File option enabled. After rebuilding, the .hex file is generated and loaded into the LPC2148 model in Proteus.

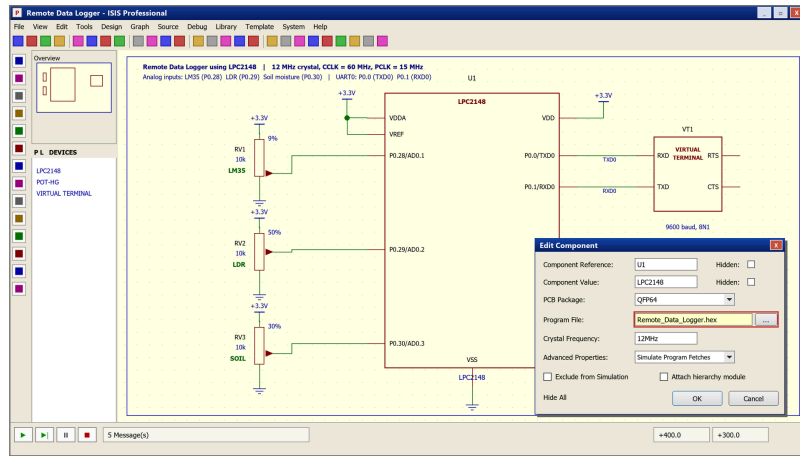


Figure 9: Simulation Setup for Remote Data Logger

Figure 9 shows the simulation circuit: the three analog sensor inputs, the 3.3 V reference and the virtual terminal connected to UART0.

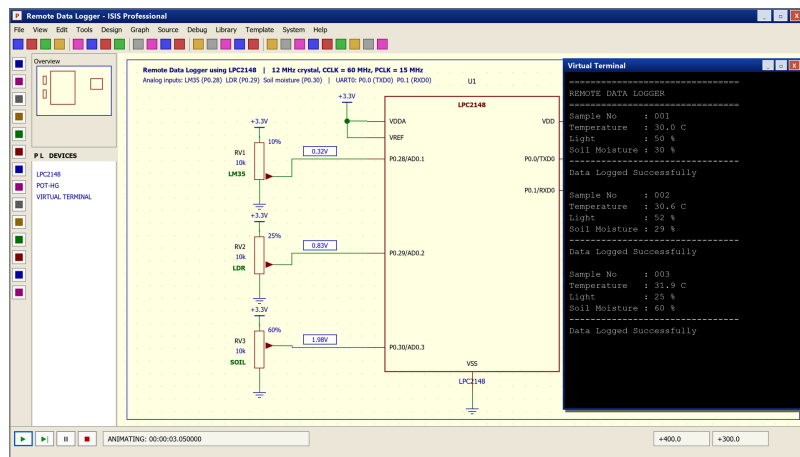


Figure 10: Remote Data Logger UART Output

Figure 10 shows the data received through UART0: the banner and the numbered sample blocks with the temperature, light and soil-moisture readings.

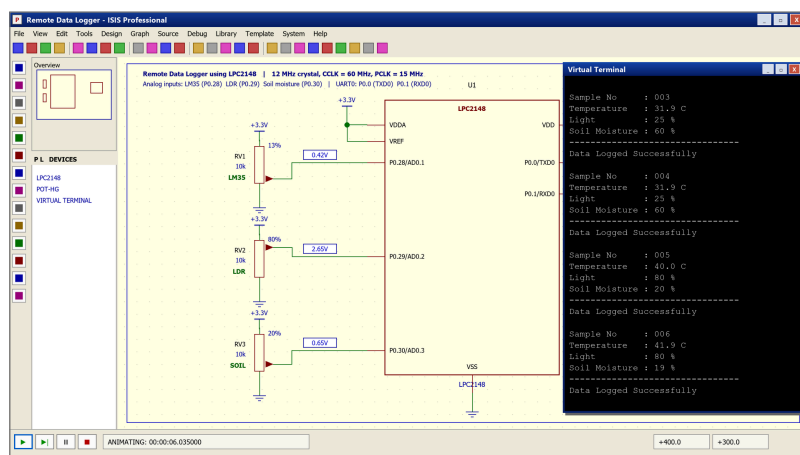


Figure 11: UART Output after Changing the Sensor Inputs

Figure 11 shows that a change in a sensor input appears in the next sample, which demonstrates that the ADC readings follow the analog inputs.

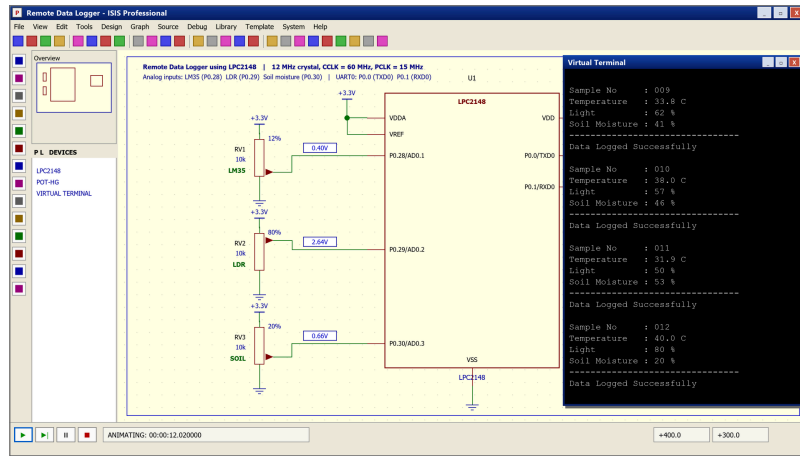


Figure 12: Final Output of Remote Data Logger

Figure 12 shows the complete system running: sensor inputs, LPC2148 and the virtual terminal displaying the logged sensor data.

Sample output. Table 9 lists the output that the program produces for the ADC counts given in the table. These values are calculated with the conversion formulas of the program and are given as sample / expected values; the values actually observed in the simulation are those shown in Figures 10 to 12.

Table 9: Sample output of the remote data logger (calculated from the stated ADC counts)

Sample	Temperature	Light	Soil Moisture	UART Status
001	30.0 °C (ADC 93)	50 % (ADC 512)	30 % (ADC 307)	Sent via UART0
002	30.6 °C (ADC 95)	52 % (ADC 540)	29 % (ADC 305)	Sent via UART0
003	31.9 °C (ADC 99)	25 % (ADC 256)	60 % (ADC 614)	Sent via UART0

6. DIAGRAMMATIC REPRESENTATION

6.1 System Block Diagram

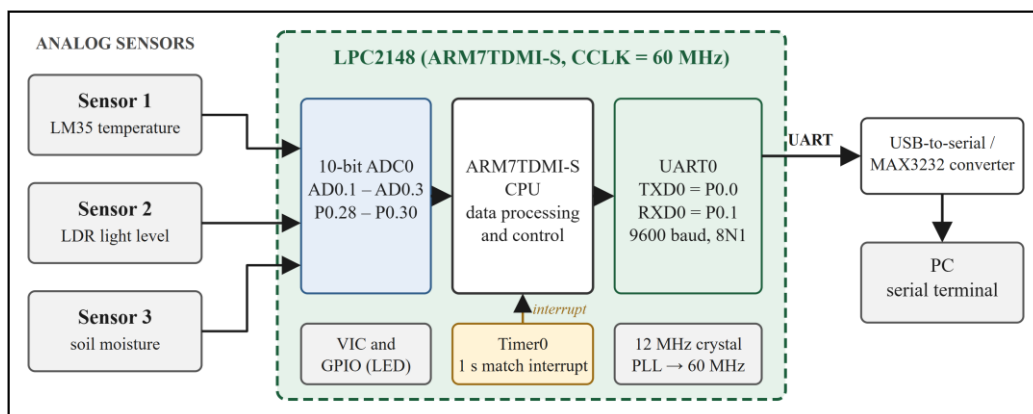


Figure 13: System Block Diagram of the Remote Data Logger

The block diagram shows the three analog sensors connected to the ADC of the LPC2148. The CPU processes the digital values, Timer0 triggers the sampling through an interrupt, and UART0 sends the data through a level converter to the PC.

6.2 Sensor-to-ADC Diagram

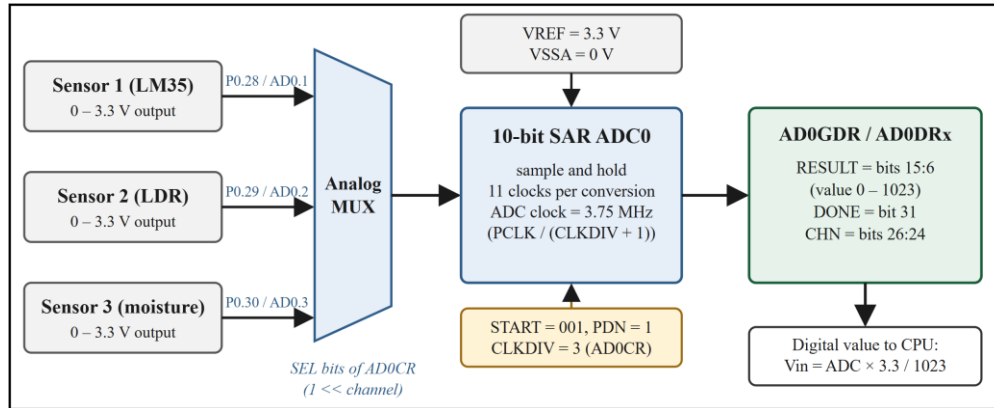


Figure 14: Sensor-to-ADC Data Acquisition in the LPC2148

The three sensor outputs enter the analog multiplexer through pins P0.28, P0.29 and P0.30. The multiplexer connects the selected channel to the 10-bit successive-approximation ADC, whose result appears in AD0GDR as a value from 0 to 1023.

6.3 UART Communication Diagram

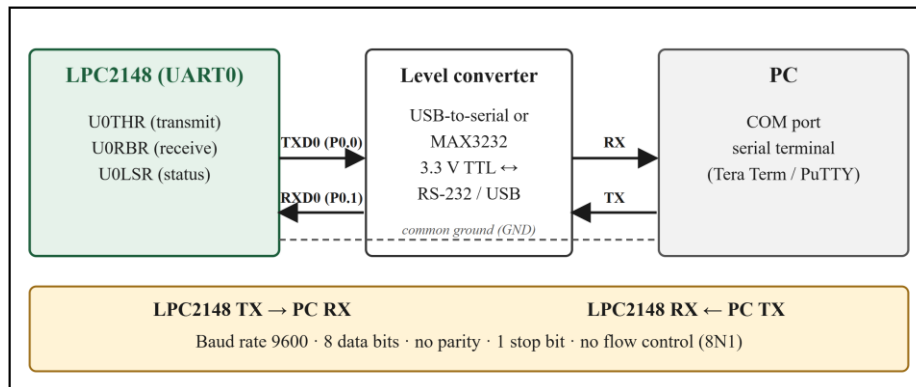


Figure 15: UART Communication between the LPC2148 and the PC (9600 baud, 8N1)

TXD0 (P0.0) of the LPC2148 is connected to the receive input of the PC, and RXD0 (P0.1) to the transmit output of the PC, with a common ground. A level converter or USB-to-serial adapter matches the 3.3 V logic levels of the LPC2148 to the PC.

6.4 Timer-Based Sampling Flowchart

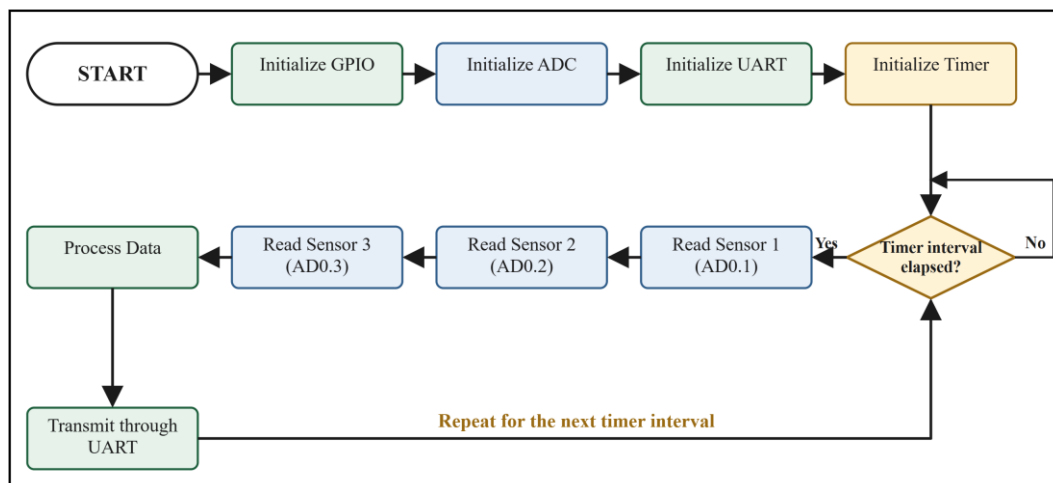


Figure 16: Timer0-Based Sampling Flowchart of the Remote Data Logger

After the GPIO, ADC, UART and Timer0 are initialised, the program waits for the Timer0 interval. When it elapses, the three sensors are read, the data is processed and transmitted through UART0, and the program waits for the next interval.

6.5 Overall System Architecture

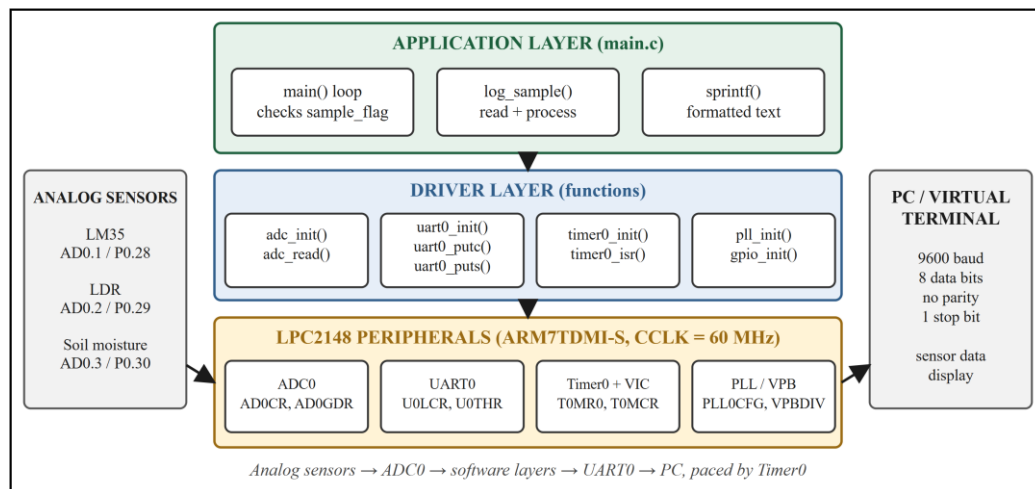


Figure 17: Overall Architecture of the Remote Data Logger

The architecture shows the three layers of the system: the application layer in main.c, the driver functions for the ADC, UART0, Timer0, PLL and GPIO, and the LPC2148 peripherals that they control. The analog sensors enter at the ADC and the PC receives the data from UART0.

7. CONCLUSION

A remote data logger was designed and implemented around the LPC2148 microcontroller. The system acquires three analog parameters (temperature, light level and soil moisture) from an LM35, an LDR and a soil-moisture sensor through the on-chip 10-bit ADC, converts the raw counts into engineering units, and sends the results to a PC through UART0 at 9600 baud, 8N1, once every second.

The **ADC** is essential because every sensor produces an analog voltage that the processor cannot use directly; its channels AD0.1 to AD0.3 allow several sensors to share one converter. The **UART0** gives a simple, reliable link to the PC, with only the TXD0 line and a level converter needed to display and store the readings. **Timer0** makes the sampling periodic and independent of the program flow, so that the interval stays accurate and the CPU is free between samples. The integration of the three modules makes the software short, deterministic and easy to extend.

The program was developed in Embedded C using Keil μ Vision: the LPC2148 project was created, main.c was added, the target was configured for a 12 MHz crystal, the project was built successfully and the HEX file was generated. In the simulation the virtual terminal displayed the sensor data at regular one-second intervals, verifying the periodic remote data logging.

Such a logger can be applied to greenhouse and agricultural monitoring, cold-storage and industrial temperature recording, weather stations and laboratory experiments. Possible future improvements are:

- replacing the wired link by a wireless one (ZigBee, GSM/GPRS or Wi-Fi);
- adding a time stamp from the on-chip real-time clock and storing the samples on an SD card;
- averaging several ADC samples, calibrating the sensors and adding alarm thresholds;
- using the remaining ADC channels for more sensors; and
- using the low-power modes of the LPC2148 between samples for battery-powered operation.